

ASSIGNMENT DSA

NAME: *Raja Sharma*

UNIVERSITY ROLL NO. : 12515001269

CLASS ROLL NO.: 46

SECTION : AD

QUESTION 1: Maximum Product of Two Elements in an Array (Leetcode no. 1464)

SOLUTION: Time Complexity $O(n)$

Accepted 104 / 104 testcases passed

Raja Sharma submitted at Aug 15, 2026 23:01

Analysis

Solution

Runtime

1 ms | Beats 59.18%

Memory

44.53 MB | Beats 58.23%



Code | Java

```
1 class Solution {
2     public int maxProduct(int[] nums) {
3
4         int largest = 0;
5         int second = 0;
6
7         for (int i = 0; i < nums.length; i++) {
8
9             if (nums[i] > largest) {
10                 second = largest;
11                 largest = nums[i];
12             }
13             else if (nums[i] > second) {
14                 second = nums[i];
15             }
16         }
17
18         return (largest - 1) * (second - 1);
19     }
20 }
```

View less

QUESTION 2: Second-Largest.

SOLUTION: Time Complexity $O(n)$

```
class Solution {  
    public int secondLargest(int[] arr) {  
        int largest = Integer.MIN_VALUE;  
        int second = Integer.MIN_VALUE;  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] > largest) {  
                second = largest;  
                largest = arr[i];  
            }  
            else if (arr[i] > second && arr[i] != largest) {  
                second = arr[i];  
            }  
        }  
        return second;  
    }  
}
```

QUESTION 3: Find Peak Element (Leetcode no. 162).

SOLUTION: Time Complexity $O(\log n)$

Accepted 76 / 76 testcases passed

Raja Sharma submitted at Aug 15, 2026 19:01

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

44.16 MB | Beats 57.14%



Code | Java

```
1 class Solution {
2     public int findPeakElement(int[] nums) {
3         int left = 0;
4         int right = nums.length - 1;
5         while (left < right) {
6             int mid = left + (right - left) / 2;
7
8             if (nums[mid] > nums[mid + 1]) {
9                 right = mid;
10            } else {
11                left = mid + 1;
12            }
13        }
14        return left;
15    }
16 }
```

View less

QUESTION 4: Search in Rotated Sorted Array (Leetcode no. 33)

SOLUTION: Time Complexity $O(\log n)$

Accepted 197 / 197 testcases passed

Raja Sharma submitted at Aug 15, 2026 23:22

Analysis

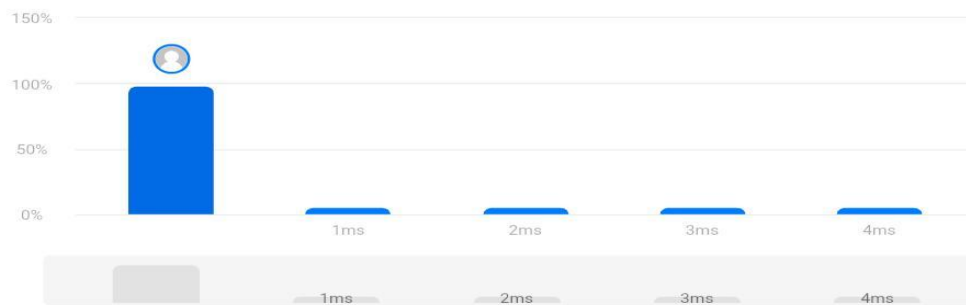
Solution

Runtime

0 ms | Beats 100.00%

Memory

43.51 MB | Beats 92.57%



Code | Java

```
1 class Solution {
2     public int search(int[] nums, int target) {
3         int low = 0;
4         int high = nums.length - 1;
5         while (low <= high) {
6             int mid = low + (high - low) / 2;
7
8             if (nums[mid] == target) {
9                 return mid;
10            }
11            if (nums[low] <= nums[mid]) {
12
13                if (target >= nums[low] && target < nums[mid]) {
14                    high = mid - 1;
15                } else {
16                    low = mid + 1;
17                }
18            }
19        }
20        else {
21
22            if (target > nums[mid] && target <= nums[high]) {
23                low = mid + 1;
24            } else {
25                high = mid - 1;
26            }
27        }
28    }
29    return -1;
30 }
31 }
```

QUESTION 5: Fruits Into Baskets II (Leetcode no.3477)

SOLUTION: Time Complexity $O(n^2)$

Accepted 904 / 904 testcases passed

Raja Sharma submitted at Aug 15, 2026 21:52

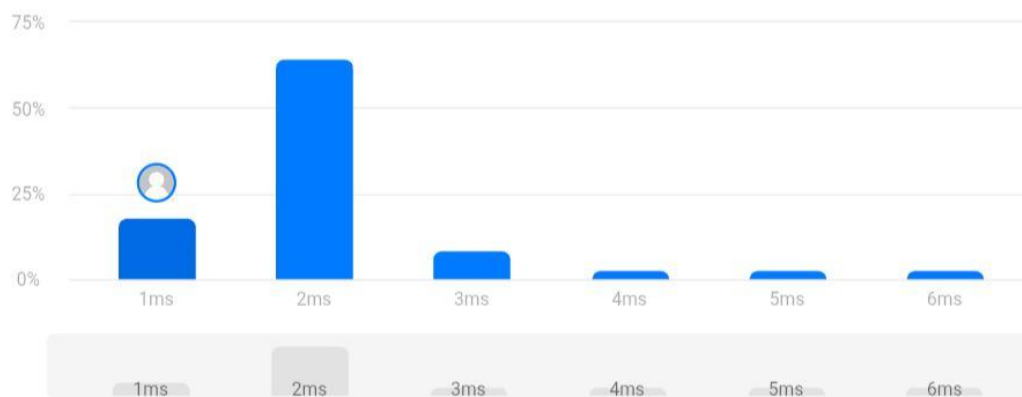
[Solution](#)

Runtime

1 ms | Beats 100.00%

Memory

46.43 MB | Beats 49.28%



Code | Java

```
1 class Solution {
2     public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
3         int count = 0;
4         for (int i = 0; i < fruits.length; i++) {
5             boolean placed = false;
6             for (int j = 0; j < baskets.length; j++) {
7                 if (baskets[j] >= fruits[i]) {
8                     baskets[j] = -1;
9                     placed = true;
10                    break;
11                }
12            }
13            if (!placed) {
14                count++;
15            }
16        }
17        return count;
18    }
19 }
```

[View less](#)

QUESTION 6: Best Time to Buy and Sell Stock (Leetcode no. 121)

SOLUTION: Time Complexity $O(n)$

Accepted 213 / 213 testcases passed

Raja Sharma submitted at Aug 15, 2026 22:49

Analysis

Solution

Runtime

2 ms | Beats 42.57%

Memory

94.36 MB | Beats 71.61%



Code | Java

```
1 class Solution {
2     public int maxProfit(int[] prices) {
3
4         int minPrice = prices[0];
5         int maxProfit = 0;
6
7         for (int i = 1; i < prices.length; i++) {
8
9             if (prices[i] < minPrice) {
10                 minPrice = prices[i];
11             }
12
13             if (prices[i] - minPrice > maxProfit) {
14                 maxProfit = prices[i] - minPrice;
15             }
16         }
17
18         return maxProfit;
19     }
20 }
```

View less

QUESTION 7: Two Sum (Leetcode no. 1)

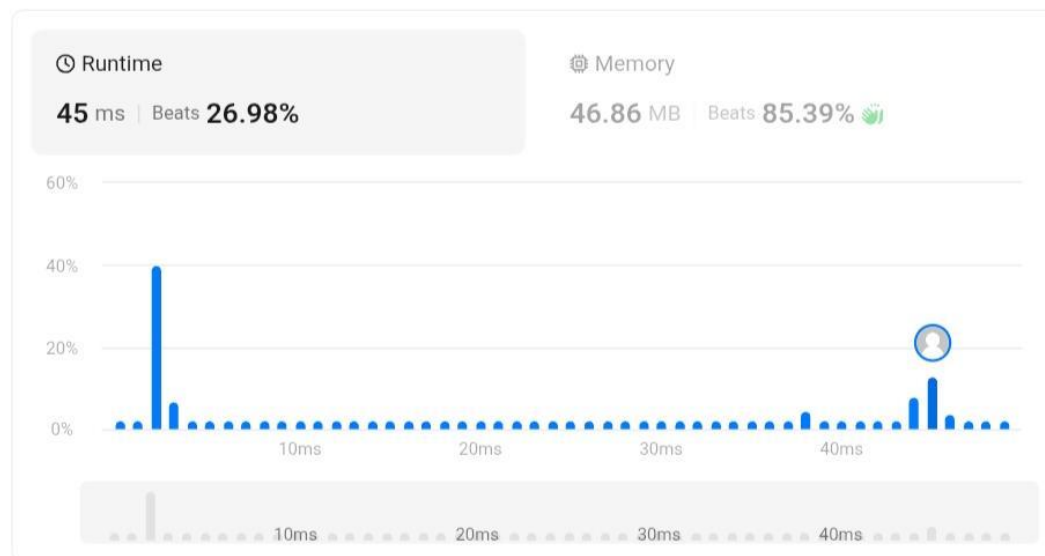
SOLUTION: Time Complexity $O(n)$

Accepted 64 / 64 testcases passed

Raja Sharma submitted at Jul 28, 2026 18:59

Analysis

Solution



Code | Java

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         for(int i=0; i<nums.length; i++){
4             for(int j=i+1; j<nums.length; j++){
5                 if(nums[i]+nums[j]==target)
6                     return new int[]{i, j};
7             }
8         }
9         return new int[]{};
10    }
11 }
```

View less

QUESTION 8: How Many Numbers Are Smaller Than the Current Number (Leetcode no 1365)

SOLUTION: Time Complexity $O(n^2)$

Accepted 103 / 103 testcases passed

Raja Sharma submitted at Aug 02, 2026 14:44

Analysis

Solution

Runtime

8 ms | Beats 63.55%

Memory

45.46 MB | Beats 68.24%



Code | Java

```
1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3         int[] array = new int[nums.length];
4         for(int i=0; i<nums.length; i++){
5             int count = 0;
6             for(int j=0; j<nums.length; j++){
7                 if((nums[i]>nums[j])){
8                     count++;
9                 }
10                array[i]=count;
11            }
12        }
13        return array;
14    }
15 }
```

View less

QUESTION 9: Maximum Product of Three Numbers (Leetcode no 628)

SOLUTION: Time Complexity $O(n)$

Accepted 93 / 93 testcases passed

Raja Sharma submitted at Aug 15, 2026 23:13

Analysis

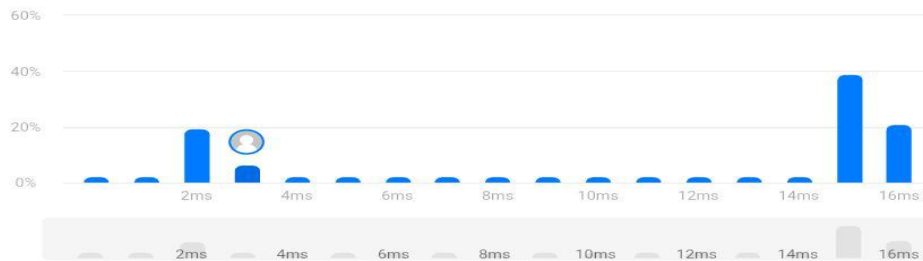
Solution

Runtime

3 ms | Beats 79.85%

Memory

46.95 MB | Beats 97.24%



Code | Java

```
1 class Solution {
2     public int maximumProduct(int[] nums) {
3
4         int max1 = Integer.MIN_VALUE;
5         int max2 = Integer.MIN_VALUE;
6         int max3 = Integer.MIN_VALUE;
7
8         int min1 = Integer.MAX_VALUE;
9         int min2 = Integer.MAX_VALUE;
10
11        for (int i = 0; i < nums.length; i++) {
12
13            if (nums[i] > max1) {
14                max3 = max2;
15                max2 = max1;
16                max1 = nums[i];
17            }
18            else if (nums[i] > max2) {
19                max3 = max2;
20                max2 = nums[i];
21            }
22            else if (nums[i] > max3) {
23                max3 = nums[i];
24            }
25
26            if (nums[i] < min1) {
27                min2 = min1;
28                min1 = nums[i];
29            }
30            else if (nums[i] < min2) {
31                min2 = nums[i];
32            }
33        }
34
35        int product1 = max1 * max2 * max3;
36        int product2 = min1 * min2 * max1;
37
38        if (product1 > product2) {
39            return product1;
40        }
41        else {
42            return product2;
43        }
44    }
45 }
```

QUESTION 10: Remove Duplicates from Sorted Array (Leetcode no 26)

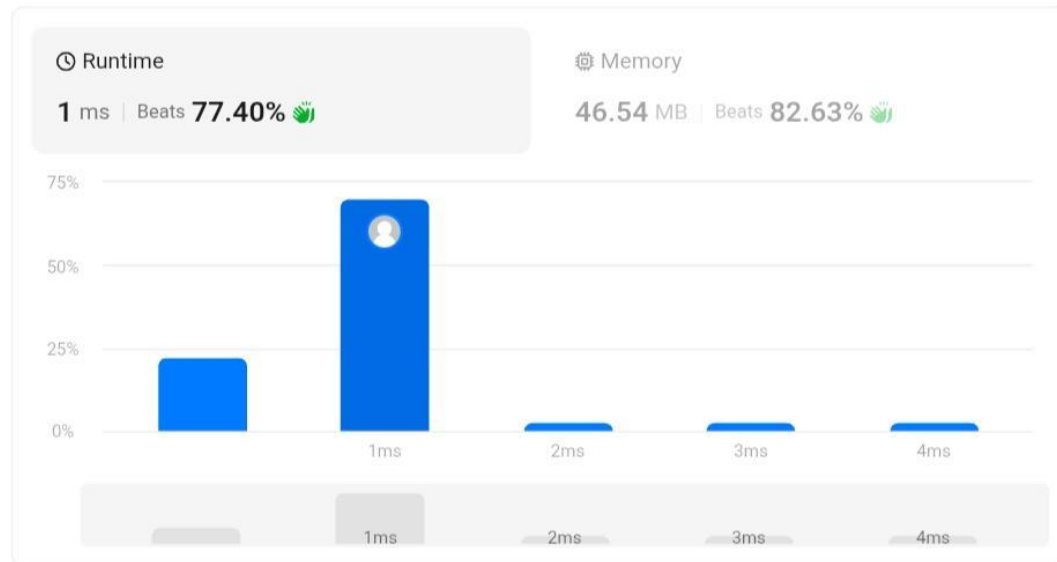
SOLUTION: Time Complexity $O(n)$

Accepted 362 / 362 testcases passed

Raja Sharma submitted at Aug 01, 2026 15:43

Analysis

Solution



Code | Java

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k=1;
4         for(int j=1;j<nums.length;j++){
5             if(nums[j]!=nums[j-1]){
6                 nums[k]=nums[j];
7                 k++;
8             }
9         }
10        return k;
11    }
12 }
```

View less